

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/1
		จำนวน 90 นาที

ชื่อ-สกุล รหัสนักศึกษา หมู่เรียน

1. จุดประสงค์ทั่วไป

เพื่อให้สามารถเขียนโปรแกรมควบคุมอุปกรณ์ไฟฟ้าด้วยการตรวจจับการเคลื่อนไหว และตรวจสอบความถูกต้องของโปรแกรมและวงจร ด้วยโปรแกรมจำลอง Wokwi ที่ทำงานร่วมกับ Visual Studio Code ก่อนนำไปติดตั้งลงบอร์ดจริง

2. จุดประสงค์เชิงพฤติกรรม

- 2.1 อธิบายหลักการทำงานของเซนเซอร์ตรวจจับการเคลื่อนไหว (PIR Sensor) ได้
- 2.2 บอกหน้าที่ของไฟล์ diagram.json, wokwi.toml และ platformio.ini ในโปรเจกต์จำลองได้
- 2.3 ติดตั้งและตั้งค่าส่วนขยาย Wokwi Simulator บน Visual Studio Code ได้
- 2.4 ต่อวงจรจำลอง NodeMCU/ESP32 ร่วมกับ PIR Sensor รีเลย์ และหลอด LED บน Wokwi ได้ถูกต้อง
- 2.5 เขียนโปรแกรมควบคุมการเปิด-ปิดอุปกรณ์ไฟฟ้าโดยการตรวจจับการเคลื่อนไหว และทดสอบผ่านโปรแกรมจำลองได้
- 2.6 วิเคราะห์ข้อผิดพลาดของโปรแกรมจากผลที่แสดงใน Serial Monitor และแก้ไขให้ทำงานถูกต้องได้

3. เครื่องมือและอุปกรณ์

- 3.1 เครื่องคอมพิวเตอร์ที่เชื่อมต่ออินเทอร์เน็ต
- 3.2 โปรแกรม Visual Studio Code (VS Code)
- 3.3 ส่วนขยาย PlatformIO IDE (สำหรับคอมไพเลอร์โค้ด)
- 3.4 ส่วนขยาย Wokwi Simulator (สำหรับจำลองวงจรและการทำงาน)
- 3.5 บัญชีผู้ใช้ Wokwi สำหรับขอ License แบบใช้งานฟรี
- 3.6 ไฟล์โปรเจกต์ตั้งต้น Workshop_5 (diagram.json, wokwi.toml, platformio.ini, src/main.cpp)

4. ลำดับขั้นตอนการปฏิบัติงาน

- 4.1 ศึกษาทฤษฎีพื้นฐานที่เกี่ยวข้อง (15 นาที)
- 4.2 เตรียมเครื่องมือและสร้างโครงสร้างโปรเจกต์ (15 นาที)
- 4.3 ต่อวงจรจำลองตามที่กำหนดไว้ใน diagram.json (15 นาที)
- 4.4 เขียนโปรแกรมควบคุมและคอมไพล์ (20 นาที)

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/2
		จำนวน 90 นาที

4.5 ทดสอบการทำงานและบันทึกผลลงตารางบันทึกผล (15 นาที)

4.6 ตอบคำถามท้ายการทดลองและสรุปผลการทดลอง (10 นาที)

เตรียมก่อนเข้าคาบเรียน

ให้นักศึกษาติดตั้ง VS Code, ส่วนขยาย PlatformIO IDE และส่วนขยาย Wokwi Simulator มาให้เรียบร้อยก่อนเข้าคาบ เนื่องจากการติดตั้ง PlatformIO ครั้งแรกต้องดาวน์โหลดชุดคอมไพเลอร์ของ ESP32 ซึ่งใช้เวลานานกว่า 10 นาที และใช้อินเทอร์เน็ตปริมาณมาก

5. ทฤษฎีพื้นฐาน

5.1 เซนเซอร์ตรวจจับการเคลื่อนไหว (PIR Sensor)

อุปกรณ์ตรวจจับคลื่นรังสีอินฟราเรด (Infrared) จากวัตถุ ผ่านอุปกรณ์รวมแสงมายังโฟโตรีเล็คทริก (Pyro Electric) ซึ่งจะเปลี่ยนพลังงานความร้อนจากรังสีอินฟราเรดเป็นพลังงานไฟฟ้า แม้จะมีปริมาณของคลื่นรังสีอินฟราเรดเพียงเล็กน้อย จึงทำให้ PIR สามารถตรวจจับคลื่นรังสีอินฟราเรดและอุณหภูมิที่แปรจากมนุษย์หรือสัตว์ที่มีการเคลื่อนไหวได้ จึงมีการนำ PIR มาประยุกต์ใช้เพื่อตรวจจับการเคลื่อนไหวของสิ่งมีชีวิต หรือตรวจจับการบุกรุกในงานรักษาความปลอดภัย

ภายใน PIR จะมีอุปกรณ์ตรวจจับรังสีอินฟราเรดอยู่ 2 ชุด เมื่อคนหรือสัตว์ที่มีความอบอุ่นในร่างกายเคลื่อนที่ผ่านเข้ามาในพื้นที่โซนที่ PIR ตรวจจับได้ PIR จะเปลี่ยนคลื่นรังสีอินฟราเรดให้กลายเป็นกระแสไฟฟ้า โดยเมื่อสิ่งมีชีวิตเคลื่อนที่ผ่านตัวตรวจจับตัวที่ 1 จะได้ค่าสัญญาณเอาต์พุตสูงกว่าค่าแรงดันปกติ และเมื่อผ่านตัวตรวจจับตัวที่ 2 จะได้ค่าแรงดันเอาต์พุตที่ต่ำกว่าค่าแรงดันปกติ

5.2 ข้อมูลเชิงเทคนิคของ PIR Sensor

5.2.1 ตรวจจับรังสีอินฟราเรด

5.2.2 ระยะในการตรวจจับ 3-7 เมตร

5.2.3 แรงดันใช้งาน DC 3-5 V

5.2.4 สัญญาณ Output 3.3V TTL

5.2.5 อุณหภูมิใช้งาน -20 ถึง 80 °C

5.2.6 ความกว้างเชิงมุมในการวัด (measuring angle) 0-140 องศา

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/3
		จำนวน 90 นาที

5.3 การจำลอง PIR Sensor ใน Wokwi

ในโปรแกรมจำลอง อุปกรณ์ PIR มีชื่อชนิดว่า wokwi-pir-motion-sensor มีขาใช้งาน 3 ขา คือ VCC, GND และ OUT เมื่อคลิกที่ตัวเซนเซอร์ขณะกำลังจำลอง จะปรากฏปุ่ม "Simulate Motion" ให้กดเพื่อจำลองว่ามีสิ่งมีชีวิตเคลื่อนที่ผ่าน ขา OUT จะเปลี่ยนเป็นลอจิก HIGH ตามเวลาที่กำหนดไว้ในคุณสมบัติ delayTime แล้วจึงกลับเป็น LOW

คุณสมบัติ (attribute)	ความหมาย	ค่าปริยาย
delayTime	จำนวนวินาทีที่ขา OUT คงสถานะ HIGH หลังตรวจพบการเคลื่อนไหว	5
inhibitTime	จำนวนวินาทีที่เซนเซอร์จะไม่รับสัญญาณใด ๆ หลังขา OUT กลับเป็น LOW	1.2
retrigger	กำหนดเป็น "0" เพื่อปิดการต่อเวลาเมื่อตรวจพบการเคลื่อนไหวซ้ำ	เปิดใช้งาน

การกำหนด delayTime มีผลเทียบเท่ากับการหมุนปรับตัวต้านทานปรับค่าได้ (potentiometer) บนเซนเซอร์ตัวจริง

5.4 โมดูลรีเลย์ในโปรแกรมจำลอง

โมดูลรีเลย์ (wokwi-relay-module) ทำหน้าที่เป็นสวิตช์ที่ควบคุมด้วยไฟฟ้า ใช้แยกวงจรควบคุมแรงดันต่ำ (3.3V) ออกจากวงจรโหลดแรงดันสูง มีขาใช้งานดังตาราง

ขา	หน้าที่
VCC	แรงดันไฟเลี้ยงของโมดูล
GND	กราวด์ร่วมกับไมโครคอนโทรลเลอร์
IN	ขารับสัญญาณควบคุมจากไมโครคอนโทรลเลอร์
COM	ขาร่วม (Common) ของหน้าสัมผัส
NO	หน้าสัมผัสปกติเปิด (Normally Open) — ปกติไม่ต่อกับ COM
NC	หน้าสัมผัสปกติปิด (Normally Closed) — ปกติต่อกับ COM

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/4
		จำนวน 90 นาที

ข้อสังเกตเรื่องทิศทางการทำงานของรีเลย์

ใบงานนี้กำหนดคุณสมบัติ transistor เป็น "pnp" เพื่อให้รีเลย์ทำงานแบบ active-high คือเมื่อขา IN เป็น HIGH หน้าสัมผัส COM จะต่อเข้ากับ NO ทำให้หลอดหลอดติด หากทดสอบแล้วพบว่าหลอดติดสลับกับที่ควรจะเป็น ให้ย้ายสายจากขา NO ไปยังขา NC หรือแก้ค่า transistor เป็น "npn" แทน

5.5 ข้อจำกัดของโปรแกรมจำลอง : บอร์ดที่ใช้แทน NodeMCU

คำเตือนสำคัญ

ปัจจุบัน Wokwi ยังไม่รองรับการจำลองชิป ESP8266 ซึ่งเป็นชิปของบอร์ด NodeMCU (สถานะเป็นเพียงคำขอเพิ่มคุณสมบัติที่ยังไม่ได้พัฒนา) ใบงานนี้จึงใช้บอร์ด ESP32 DevKit V1 เป็นตัวแทนในการจำลอง โดยหมายเลข GPIO เดิมของใบงานไว้ทั้งหมด (GPIO16, GPIO12 และ GPIO13) เพื่อให้โค้ดที่ทดสอบแล้วสามารถย้ายไปใช้กับบอร์ดจริงได้โดยแก้ไขน้อยที่สุด

ตารางเทียบขาระหว่างบอร์ดจริงกับบอร์ดจำลอง

อุปกรณ์	ขาบน NodeMCU (ใบงานเดิม)	หมายเลข GPIO	ชื่อขาบน ESP32 DevKit V1
LED แสดงสถานะ	D0	16	RX2
ขาควบคุมรีเลย์ (IN)	D6	12	D12
สัญญาณจาก PIR (OUT)	D7	13	D13
ไฟเลี้ยง 3.3V	3V3	-	3V3
ไฟเลี้ยง 5V	VU / VIN	-	VIN
กราวด์	GND	-	GND.1 / GND.2

บนบอร์ด ESP32 DevKit V1 ขา GPIO16 ถูกพิมพ์ชื่อกำกับไว้ว่า RX2 แต่เมื่อไม่ได้ใช้งานเป็นพอร์ตอนุกรมที่สอง สามารถใช้เป็นขาดิจิทัลทั่วไปได้ตามปกติ ส่วนขา GPIO12 บนบอร์ดจริงเป็นขากำหนดค่าเริ่มต้นของระบบ (strapping pin) ควรหลีกเลี่ยงการต่อสัญญาณค้างไว้ขณะเปิดเครื่อง แต่ในโปรแกรมจำลองไม่มีผลกระทบดังกล่าว

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/5
		จำนวน 90 นาที

6. การเตรียมเครื่องมือและสร้างโปรเจกต์

6.1 ติดตั้งส่วนขยายที่จำเป็น

- 1) เปิด VS Code แล้วไปที่แถบ Extensions (Ctrl+Shift+X)
- 2) ค้นหาคำว่า platformio แล้วติดตั้ง PlatformIO IDE รอจนติดตั้งเสร็จและรีสตาร์ท VS Code
- 3) ค้นหาคำว่า wokwi แล้วติดตั้ง Wokwi Simulator (ผู้พัฒนาคือ Wokwi)

6.2 ขอรับ License สำหรับใช้งาน

- 1) กด Ctrl+Shift+P เพื่อเปิด Command Palette
- 2) พิมพ์ wokwi แล้วเลือกคำสั่ง Wokwi: Request a New License
- 3) กดปุ่ม Open เพื่อเปิดเว็บเบราว์เซอร์ แล้วกด Get Your License
- 4) ยืนยันให้เบราว์เซอร์เปิดกลับมายัง VS Code ระบบจะบันทึก License ให้อัตโนมัติ

6.3 สร้างโครงสร้างไฟล์ของโปรเจกต์

สร้างโฟลเดอร์ชื่อ Workshop_5 แล้วเปิดด้วยคำสั่ง File → Open Folder... จากนั้นสร้างไฟล์ให้ครบตามโครงสร้างต่อไปนี้

Workshop_5/		
—	platformio.ini	ตั้งค่าบอร์ดและ framework
—	wokwi.toml	บอก Wokwi ว่าไฟล์เฟิร์มแวร์อยู่ที่ใด
—	diagram.json	วงจรจำลอง
—	src/	
—	main.cpp	โค้ดโปรแกรม

6.4 ไฟล์ platformio.ini

1	[env:esp32doit-devkit-v1]	
2	Platform	= espressif32
3	board	= esp32doit-devkit-v1
4	framework	= arduino
5	monitor_speed	= 115200

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจัดการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/6
		จำนวน 90 นาที

6.5 ไฟล์ wokwi.toml

1	[wokwi]
2	version = 1
3	firmware = '.pio/build/esp32doit-devkit-v1/firmware.bin'
4	elf = '.pio/build/esp32doit-devkit-v1/firmware.elf'

ไฟล์นี้ทำหน้าที่บอกส่วนขยาย Wokwi ว่าไฟล์เฟิร์มแวร์ที่คอมไพล์เสร็จแล้วถูกเก็บไว้ที่ใด หากพารามิเตอร์
ถูกต้อง โปรแกรมจำลองจะไม่สามารถเริ่มทำงานได้

7. วงจรการทดลอง

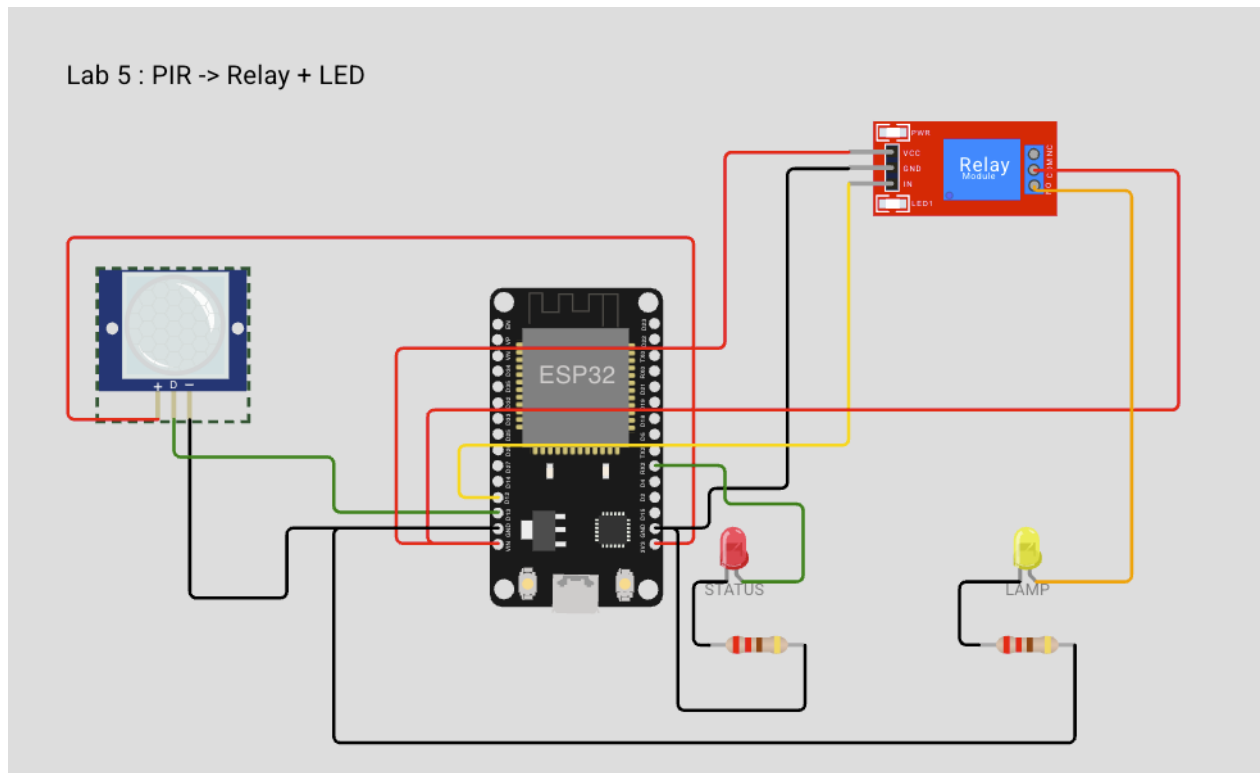
ต่อวงจรตามตารางต่อไปนี้ โดยพิมพ์เนื้อหาของไฟล์ diagram.json ในหัวข้อ 7.2 ลงในโปรเจกต์

7.1 ตารางการต่อวงจร

จาก	ไปยัง	สีสาย	หน้าที่
PIR : VCC	ESP32 : 3V3	แดง	จ่ายไฟเลี้ยงเซนเซอร์
PIR : GND	ESP32 : GND.2	ดำ	กราวด์ร่วม
PIR : OUT	ESP32 : D13	เขียว	ส่งสัญญาณตรวจจับเข้า GPIO13
Relay : VCC	ESP32 : VIN	แดง	ไฟเลี้ยงโมดูลรีเลย์ 5V
Relay : GND	ESP32 : GND.1	ดำ	กราวด์ร่วม
Relay : IN	ESP32 : D12	เหลือง	รับสัญญาณควบคุมจาก GPIO12
ESP32 : RX2	LED1 (STATUS) : A	เขียว	ขับ LED แสดงสถานะจาก GPIO16
LED1 : C	R1 (220Ω) → GND.1	ดำ	จำกัดกระแสและลงกราวด์
Relay : COM	ESP32 : VIN	แดง	จ่ายไฟให้วงจรหลอด
Relay : NO	LED2 (LAMP) : A	ส้ม	หน้าสัมผัสจ่ายไฟให้หลอดหลอด
LED2 : C	R2 (220Ω) → GND.2	ดำ	จำกัดกระแสและลงกราวด์

LED2 (LAMP) ใช้แทนหลอดไฟฟ้าแรงดันสูงที่ต่อผ่านหน้าสัมผัสของรีเลย์ในวงจรจริง

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/7
		จำนวน 90 นาที



ภาพที่ 1 ภาพวงจร

7.2 ไฟล์ diagram.json

สามารถคัดลอกเนื้อหาต่อไปนี้ลงในไฟล์ diagram.json ได้โดยตรง หรือจะจัดวางอุปกรณ์เองในหน้าต่างแก้ไขวงจรของ Wokwi ก็ได้

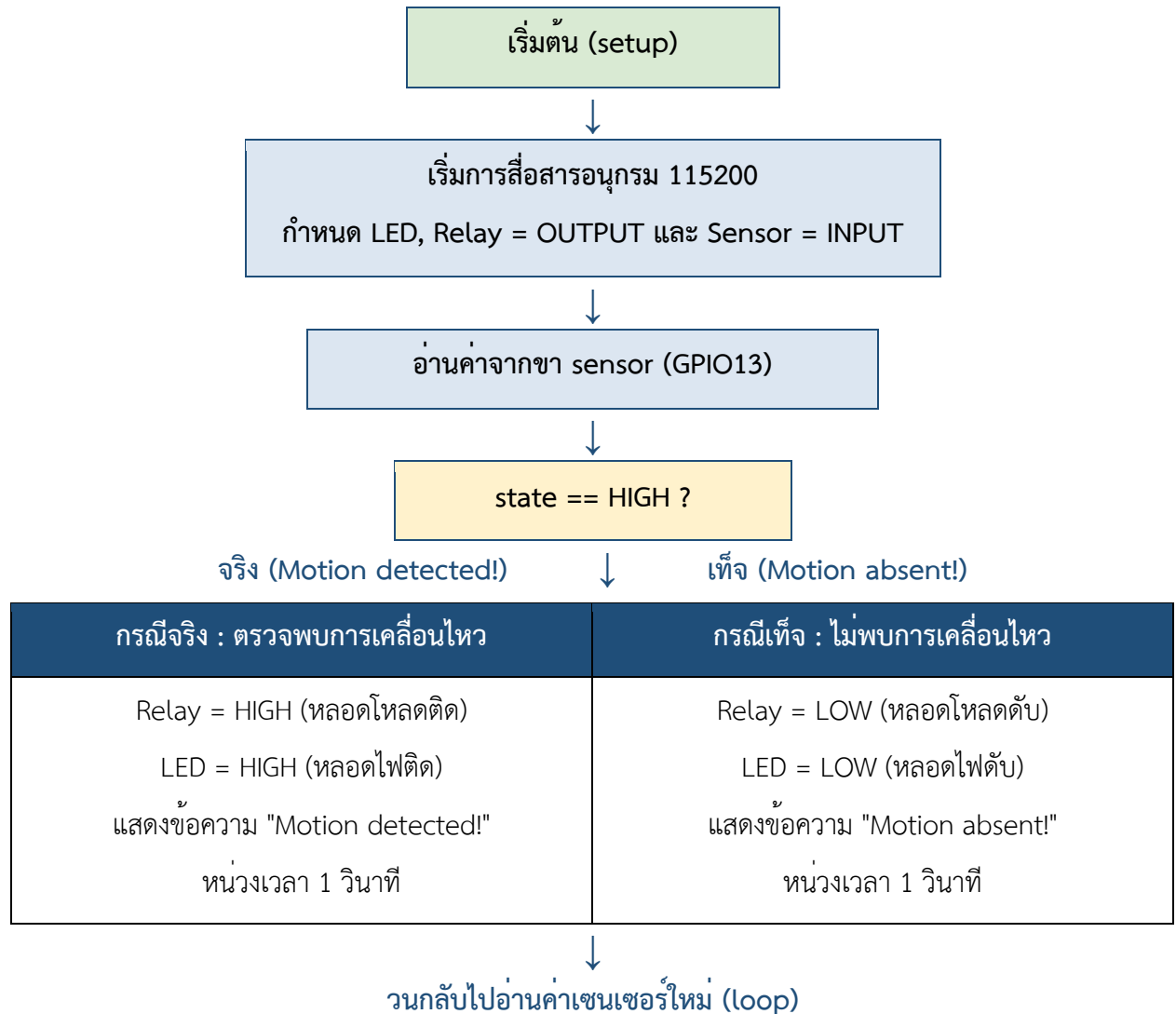
{
"version": 1, "author": "RERU-IT", "editor": "wokwi",
"parts": [
{ "type": "wokwi-esp32-devkit-v1", "id": "esp",
"top": 0, "left": 0, "attrs": {} },
{ "type": "wokwi-pir-motion-sensor", "id": "pir1",
"top": -9.4, "left": -240, "attrs": { "delayTime": "5", "inhibitTime": "1.2" } },
{ "type": "wokwi-relay-module", "id": "relay1",
"top": -100, "left": 220, "attrs": { "transistor": "pnp" } },
{ "type": "wokwi-led", "id": "led1",
"top": 140, "left": 130, "attrs": { "color": "red", "label": "STATUS" } },
{ "type": "wokwi-resistor", "id": "r1",

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/8
		จำนวน 90 นาที

"top": 215, "left": 120, "attrs": { "value": "220" } },
{ "type": "wokwi-led", "id": "led2",
"top": 140, "left": 310, "attrs": { "color": "yellow", "label": "LAMP" } },
{ "type": "wokwi-resistor", "id": "r2",
"top": 215, "left": 300, "attrs": { "value": "220" } },
{ "type": "wokwi-text", "id": "txt1",
"top": -140, "left": -260, "attrs": { "text": "Lab 5 : PIR -> Relay + LED", "fontSize": "18" } }
],
"connections": [
["esp:TX0", "\$serialMonitor:RX", "", []],
["esp:RX0", "\$serialMonitor:TX", "", []],
["pir1:VCC", "esp:3V3", "red", ["v-40", "h180"]],
["pir1:GND", "esp:GND.2", "black", ["v40", "h160"]],
["pir1:OUT", "esp:D13", "green", ["v0", "h120"]],
["relay1:VCC", "esp:VIN", "red", ["v120", "h-120"]],
["relay1:GND", "esp:GND.1", "black", ["v140", "h-140"]],
["relay1:IN", "esp:D12", "gold", ["v160", "h-200"]],
["esp:RX2", "led1:A", "green", ["h20", "v100"]],
["led1:C", "r1:1", "black", ["v20"]],
["r1:2", "esp:GND.1", "black", ["v40", "h-60"]],
["relay1:COM", "esp:VIN", "red", ["v180", "h-220"]],
["relay1:NO", "led2:A", "orange", ["v60", "h20"]],
["led2:C", "r2:1", "black", ["v20"]],
["r2:2", "esp:GND.2", "black", ["v60", "h-260"]]
],
"dependencies": {}
}

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/9
		จำนวน 90 นาที

8. ผังงาน (Flow chart) อธิบายการทำงานของโปรแกรม



ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/10
		จำนวน 90 นาที

9. การเขียนโปรแกรม

พิมพ์โปรแกรมต่อไปนี้ลงในไฟล์ src/main.cpp

1	/* -----
2	* File : main.cpp (Workshop_5)
3	* ใบงานที่ 5 : การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว
4	* ฉบับจำลองการทำงานด้วย Wokwi + Visual Studio Code
5	* บอร์ด : ESP32 DevKit V1 (ใช้แทน NodeMCU ESP8266 เนื่องจาก
6	* Wokwi ยังไม่รองรับการจำลองชิป ESP8266)
7	* คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยราชภัฏร้อยเอ็ด
8	* ----- */
9	#include <Arduino.h>
10	
11	#define LED 16 // GPIO16 (ขา RX2 บนบอร์ด) -> LED แสดงสถานะ [เดิม D0 ของ NodeMCU]
12	int Status = 12; // GPIO12 (ขา D12) -> IN ของรีเลย์ [เดิม D6 ของ NodeMCU]
13	int sensor = 13; // GPIO13 (ขา D13) <- OUT ของ PIR [เดิม D7 ของ NodeMCU]
14	
15	void setup() {
16	Serial.begin(115200); // *** ต้นฉบับใบงานไม่มีบรรทัดนี้ ถ้าไม่ใส่จะไม่เห็นข้อความใน Serial Monitor ***
17	pinMode(LED, OUTPUT); // กำหนด LED เป็น output
18	pinMode(sensor, INPUT); // กำหนด sensor เป็น input
19	pinMode(Status, OUTPUT); // กำหนดขาควบคุมรีเลย์เป็น output
20	digitalWrite(LED, LOW);
21	digitalWrite(Status, LOW);
22	Serial.println("Workshop 5 : PIR Motion Detection -> Relay + LED");
23	}

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/11
		จำนวน 90 นาที

24	
25	void loop() {
26	long state = digitalRead(sensor); // อ่านค่าจากเซนเซอร์ PIR
27	
28	if (state == HIGH) {
29	digitalWrite(Status, HIGH); // สั่งรีเลย์ทำงาน (หลอดโหลดติด)
30	Serial.println("Motion detected!"); // ตรวจพบการเคลื่อนไหวของมนุษย์
31	digitalWrite(LED, HIGH); // หลอดไฟติด
32	delay(1000);
33	}
34	else {
35	digitalWrite(Status, LOW); // สั่งรีเลย์หยุดทำงาน (หลอดโหลดดับ)
36	Serial.println("Motion absent!"); // ไม่มีการเคลื่อนไหว
37	digitalWrite(LED, LOW); // หลอดไฟดับ
38	delay(1000);
39	}
40	}

ข้อสังเกตที่ 1 : บรรทัด Serial.begin() ที่หายไป

โปรแกรมต้นฉบับในใบงานเดิมเรียกใช้คำสั่ง Serial.println() แต่ไม่ได้เรียก Serial.begin() ในฟังก์ชัน setup() จึงทำให้ไม่ปรากฏข้อความใด ๆ บนหน้าจอ Serial Monitor ใบงานฉบับนี้จึงเพิ่มบรรทัด Serial.begin(115200) เข้าไป และต้องตั้งค่าความเร็วของ Serial Monitor ให้ตรงกันด้วย

ข้อสังเกตที่ 2 : ผลของคำสั่ง delay(1000)

คำสั่ง delay() จะหยุดการทำงานของโปรแกรมทั้งหมดชั่วคราว ทำให้ในช่วง 1 วินาทีนั้นโปรแกรมไม่สามารถอ่านค่าเซนเซอร์หรือตอบสนองต่อเหตุการณ์อื่นได้เลย
ประเด็นนี้จะนำไปพัฒนาต่อในกิจกรรมท้าทายข้อที่ 3

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/12
		จำนวน 90 นาที

10. การทดสอบ

- 10.1 กดปุ่ม Build (เครื่องหมายลูกศร) บนแถบสถานะของ PlatformIO หรือใช้คำสั่ง pio run รอจนขึ้นข้อความ SUCCESS
- 10.2 กด Ctrl+Shift+P แล้วเลือกคำสั่ง Wokwi: Start Simulator
- 10.3 สังเกตหน้าต่างจำลอง ให้ปรากฏบอร์ด ESP32 เซนเซอร์ PIR โมดูลรีเลย์ และหลอด LED ครบถ้วน
- 10.4 คลิกที่เซนเซอร์ PIR ขณะกำลังจำลอง แล้วกดปุ่ม Simulate Motion
- 10.5 สังเกตการทำงานของหลอด LED ทั้งสองดวง เสียงและสถานะของรีเลย์ และข้อความบน Serial Monitor
- 10.6 บันทึกผลลงในตารางบันทึกผลการทดลอง

ตารางบันทึกผลการทดลอง

ครั้งที่	เหตุการณ์ที่ทดสอบ	สถานะ LED (STATUS)	สถานะหลอด (LAMP)	ข้อความบน Serial Monitor
1	ก่อนกด Simulate Motion			
2	ทันทีที่กด Simulate Motion			
3	หลังกด Simulate Motion 3 วินาที			
4	หลังกด Simulate Motion 7 วินาที			
5	แก้ delayTime เป็น 10 แล้วทดสอบซ้ำ			

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/13
		จำนวน 90 นาที

11. คำถามท้ายการทดลอง

11.1 เพราะเหตุใดจึงต้องใช้บอร์ด ESP32 แทน NodeMCU ในการทดลองด้วยโปรแกรมจำลองนี้ และมีผลต่อโค้ดอย่างไรบ้าง

11.2 ไฟล์ wokwi.toml และ diagram.json ทำหน้าที่ต่างกันอย่างไร ถ้าวัดไฟล์ใดไฟล์หนึ่งออกจะเกิดผลอย่างไร

11.3 จากตารางบันทึกผล เหตุใดข้อความ "Motion detected!" จึงยังคงแสดงต่อเนื่องแม้จะกด Simulate Motion เพียงครั้งเดียว

11.4 อธิบายหน้าที่ของขา COM, NO และ NC ของรีเลย์ และเหตุใดจึงต้องใช้รีเลย์แทนการต่อหลอดไฟเข้ากับขา GPIO โดยตรง

11.5 หากต้องการให้หลอดไฟติดค้างไว้ 10 วินาทีหลังตรวจพบการเคลื่อนไหวครั้งสุดท้าย จะต้องแก้ไขส่วนใดของโปรแกรมหรือของวงจรจำลอง

11.6 ระบุข้อดีและข้อจำกัดของการทดสอบด้วยโปรแกรมจำลอง เมื่อเทียบกับการทดลองบนบอร์ดจริง อย่างละ 2 ข้อ

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/14
		จำนวน 90 นาที

12. กิจกรรมต่อยอด (เลือกทำอย่างน้อย 1 ข้อ)

12.1 ระดับต้น : เพิ่มหลอด LED สีเขียวเข้าไปในวงจร ให้ติดเมื่อไม่พบการเคลื่อนไหว และดับเมื่อพบการเคลื่อนไหว

12.2 ระดับกลาง : แก้ไขโปรแกรมให้แสดงข้อความบน Serial Monitor เฉพาะตอนที่สถานะเปลี่ยนแปลงเท่านั้น (แสดง "Motion detected!" เมื่อเปลี่ยนจากไม่พบเป็นพบ และ "Motion ended!" เมื่อเปลี่ยนกลับ) โดยใช้ตัวแปรเก็บสถานะเดิม

12.3 ระดับสูง : ปรับโปรแกรมให้เลิกใช้คำสั่ง delay() แล้วเปลี่ยนไปใช้ฟังก์ชัน millis() เพื่อหน่วงเวลาแบบไม่หยุดการทำงาน พร้อมกำหนดให้หลอดไฟติดค้างต่ออีก 10 วินาทีนับจากการตรวจจับครั้งสุดท้าย

13. ปัญหาที่พบบ่อยและแนวทางแก้ไข

อาการ	สาเหตุที่เป็นไปได้	แนวทางแก้ไข
สั่ง Start Simulator แล้วขึ้นข้อความว่าหาไฟล์เฟิร์มแวร์ไม่พบ	ยังไม่ได้ Build หรือพารามิเตอร์ใน wokwi.toml ไม่ตรงกับผลลัพธ์จริง	กด Build ให้ขึ้น SUCCESS ก่อน แล้วตรวจสอบชื่อโฟลเดอร์ใน .pio/build/ ให้ตรงกับที่ระบุใน wokwi.toml
ขึ้นข้อความเกี่ยวกับ License หมดอายุหรือไม่ถูกต้อง	ยังไม่ได้ขอ License หรือ License หมดอายุ	ใช้คำสั่ง Wokwi: Request a New License แล้วทำตามขั้นตอนในหัวข้อ 6.2 อีกครั้ง
ไม่มีข้อความใด ๆ ปรากฏบน Serial Monitor	ไม่ได้เรียก Serial.begin() หรือความเร็วไม่ตรงกัน	ตรวจสอบว่ามี Serial.begin(115200) ใน setup() และไม่ได้ลบสายที่ต่อกับ \$serialMonitor ใน diagram.json

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/15
		จำนวน 90 นาที

อาการ	สาเหตุที่เป็นไปได้	แนวทางแก้ไข
หลอด LAMP ติดตอนไม่มีการเคลื่อนไหว และดับเมื่อมีการเคลื่อนไหว	ทิศทางการทำงานของรีเลย์กลับด้าน	ย้ายสายจากขา NO ไปยัง NC หรือแก้ค่า transistor ใน diagram.json
กด Simulate Motion แล้วไม่มีอะไรเปลี่ยนแปลง	ต่อขา OUT ผิดขา หรืออยู่ในช่วง inhibit time	ตรวจสอบว่า OUT ต่อกับ D13 และรอสายอย่างน้อย 1.2 วินาทีหลังสถานะกลับเป็น LOW
คอมไพล์ไม่ผ่าน แจ้งว่าไม่รู้จักฟังก์ชันของ Arduino	ลืมบรรทัด #include <Arduino.h> ในไฟล์ .cpp	เพิ่มบรรทัด #include <Arduino.h> ไว้บรรทัดแรกของไฟล์

ใบงานที่ 5	การควบคุมอุปกรณ์ไฟฟ้า โดยการตรวจจับการเคลื่อนไหว (Wokwi + VS Code)	หน้า 5/16
		จำนวน 90 นาที

14. เกณฑ์การให้คะแนน

รายการประเมิน	คะแนน
สร้างโครงสร้างโปรเจกต์และตั้งค่าไฟล์ wokwi.toml / platformio.ini ได้ถูกต้อง คอมไพล์ผ่าน	2
ต่อวงจรจำลองครบถ้วนถูกต้องตามตารางการต่อวงจร	2
โปรแกรมทำงานได้ตามเงื่อนไข ควบคุม LED และรีเลย์ได้ถูกต้อง	2
บันทึกผลการทดลองครบทุกกรณีและสอดคล้องกับผลที่เกิดขึ้นจริง	2
ตอบคำถามท้ายการทดลองได้ถูกต้อง มีการอธิบายเหตุผลประกอบ	1
ทำกิจกรรมต่อยอดสำเร็จอย่างน้อย 1 ข้อ	1
รวม	10

15. สรุปผลการทดลอง

.....